

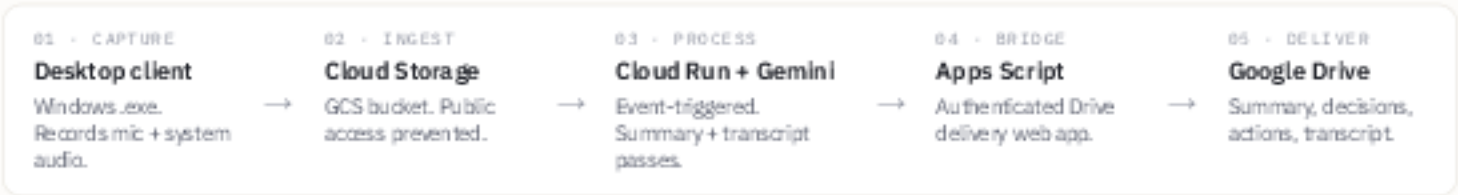
Tamlelan

Autonomous AI meeting agent · private, pay-per-use cloud

In active daily personal use

no manual transcription · pay-per-use cloud billing

A Windows desktop client records a meeting locally and uploads the audio to Cloud Storage. An Eventarc-triggered Cloud Run function runs it through the Gemini API, and structured results - executive summary, decisions log, action-item table, full verbatim transcript, and a conditional architecture diagram - are delivered to the author's Google Drive, fully automated. It runs on pay-per-use Google Cloud billing for its Gemini API calls, Cloud Run invocations, and storage, averaging roughly half the cost of a comparable subscription-based meeting-transcription tool.



READ EACH UNIT AS Problem → Decision → Why it matters

Pay-per-use cost model

GOOGLE CLOUD BILLING

PROBLEM

Dedicated AI meeting-transcription tools are typically sold as a fixed monthly subscription, priced for average usage regardless of how much is actually used.

DECISION

Build directly on Google Cloud's pay-per-use pricing for exactly the components used - Gemini API calls, Cloud Run invocations, and storage.

WHY IT MATTERS

≈ half

the cost of a comparable subscription-based tool on average - a direct, measurable build-vs-buy outcome.

Dual-audio capture

DESKTOP CLIENT

PROBLEM

Most meetings run over a call, where other participants' voices come through the speakers, not the mic. A mic-only recorder misses half the conversation.

DECISION

Capture and mix both audio paths at record time - the microphone and the system output (WASAPI loopback) - into one track.

WHY IT MATTERS

Completeness is the whole value of the recording - a transcript that captures only half the room creates false confidence.

Event-driven processing

EVENTARC + CLOUD RUN

PROBLEM

An earlier iteration (still in repo history) used a script that polled a folder on a fixed interval - idle cost, latency, and interval-miss bugs.

DECISION

Replace polling with an event trigger fired the moment a file lands in storage.

WHY IT MATTERS

Lower latency, lower idle cost, and it removes an entire class of missed-poll-interval bugs - a deliberate architectural upgrade.

Idempotency by atomic lock

PROCESSING

PROBLEM

Eventarc's at-least-once delivery is a documented platform guarantee - duplicate invocations for the same event are expected behavior.

DECISION

Before processing, create a lock object keyed to the event ID via a conditional write that only succeeds if it does not already exist; a duplicate loses the race and exits cleanly.

WHY IT MATTERS

The system is designed against the platform's actual delivery guarantees.

CORE BSA EXERCISE

Security audit and remediation

BEFORE

broad access, plaintext, open webhook

→

AFTER

scoped, secret-managed, gated

The system initially ran with a default broadly-permissioned identity, plaintext keys in configuration, an unauthenticated delivery webhook, and a client that embedded its own credentials inside the compiled binary. Each risk was identified, isolated, and remediated individually, with verification at each step - a structured gap-analysis and prioritization process applied to a system already in daily use.

01

Default broad-permission identity

→ Dedicated least-privilege service accounts, each scoped to a single resource.

02

Plaintext API keys and secrets in config

→ Moved into Secret Manager; nothing hardcoded.

03

Unauthenticated delivery webhook

→ Shared secret plus destination-folder allowlist carried in the request body (Apps Script blocks custom headers, which shaped the fix).

04

Credentials embedded in the .exe

→ Credentials file shipped alongside the binary, keeping them out of the compiled executable.

05

Broad network exposure

→ Processing service restricted to internal-only ingress.

✓

Verified at each step

Remediation ran as identify → prioritize → fix → validate, with verification confirmed at each stage.

PROCESSING

Two Gemini passes

Problem.

One prompt for both a rigid JSON structure and an unbounded transcript risks the model degrading one to satisfy the other.

Decision.

Split into two independent calls with two independent output contracts.

Why.

Each output is dependable on its own terms - a reliability trade-off made deliberately for a little added latency and token spend.

PROCESSING

Conditional diagram

Problem.

Not every meeting is worth diagramming; doing it every time is wasted cost and noise.

Decision.

The summary pass decides if a diagram is warranted; only then a lighter model (Gemini Flash) generates a Mermaid.js diagram.

Why.

Cost and relevance are treated as first-class design constraints.

DELIVERY

Apps Script bridge

Problem.

Writing output straight into free-tier Drive hit a hard storage-quota limit.

Decision.

Route final delivery through a small Apps Script web app running under the author's own Drive quota.

Why.

A real infrastructure constraint met with a pragmatic, bounded workaround - recognizing and solving for it is the point.

SCOPE

Mobile client dropped

Problem.

A second client platform (Flutter/Android) was on the original road map.

Decision.

Scope it out; redirect effort into hardening the one client and pipeline already in use.

Why.

Effort went into depth on the existing surface - a deliberate scope-prioritization decision.